

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/370009165>

Application of Metacognitive Planning Scaffolding for the Cultivation of Computational Thinking

Article in Journal of Educational Computing Research · April 2023

DOI: 10.1177/07356331231160294

CITATIONS

2

READS

148

7 authors, including:



Ying Zhou

Beijing Normal University

13 PUBLICATIONS 51 CITATIONS

SEE PROFILE



Ching Sing Chai

The Chinese University of Hong Kong

282 PUBLICATIONS 13,409 CITATIONS

SEE PROFILE



Xiuting Li

Beijing Normal University

5 PUBLICATIONS 7 CITATIONS

SEE PROFILE



Ding Yu

Beijing Normal University

2 PUBLICATIONS 2 CITATIONS

SEE PROFILE

Application of Metacognitive Planning Scaffolding for the Cultivation of Computational Thinking

Journal of Educational Computing Research
2023, Vol. 0(0) 1–20
© The Author(s) 2023
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: [10.1177/07356331231160294](https://doi.org/10.1177/07356331231160294)
journals.sagepub.com/home/jec


Ying Zhou¹ , Ching Sing Chai², Xiuting Li¹, Chao Ma¹,
Baoping Li¹, Ding Yu¹, and Jyh-Chong Liang³ 

Abstract

Computational thinking is a way of thinking that helps people “think like a computer scientist” to solve practical problems. However, practicing computational thinking through programming is dependent on the problem solvers’ metacognition. This study investigated students’ metacognitive planning and problem-solving performance in programming through two quantitative studies. First, we analyzed the performance of metacognitive planning and of problem solving through the programming of 21 freshmen, and found that the metacognitive planning performance related to “problem description” and “program comprehension” was significantly correlated with problem-solving performance. Second, semi-scaffolding and full-scaffolding were designed based on the first study. Another 89 freshmen were randomly divided into three groups and were asked to write their programming plan with no-scaffolding, semi-scaffolding, or with full-scaffolding. ANCOVA revealed that the problem-solving performance of the no-scaffolding group was significantly weaker than that of the other two groups, but there was no significant difference between the semi-scaffolding and the full-scaffolding groups. The study indicated that semi-scaffolding had a similar effect to full-scaffolding on problem-solving performance. The study suggests that

¹Faculty of Education, Beijing Normal University, Beijing, China

²Department of Curriculum and Instruction, The Chinese University of Hong Kong, Hong Kong, China

³Program of Learning Sciences and Institute for Research Excellence in Learning Sciences, National Taiwan Normal University, Taipei, Taiwan

Corresponding Author:

Ying Zhou, Faculty of Education, Beijing Normal University, No. 19, Xijiekouwai St, Haidian District, Beijing 100875, China.

Email: yzh@bnu.edu.cn

teachers should emphasize supporting students' "problem description" and "program comprehension" using semi-scaffolding. This scaffolding technique is sufficient and efficient for training students' computational thinking through problem solving in programming.

Keywords

computational thinking, problem solving through programming, metacognitive planning, scaffolding, experimental study

Introduction*Computational Thinking*

Since Wing (2006) proposed the term Computational Thinking (CT), it has received extensive attention. Computational thinking refers to the processes of "thinking like a computer scientist" that are involved in "solving problems, designing systems, and understanding human behavior, by drawing on the concepts fundamental to computer science" (Wing, 2006, p. 33). Although there are several definitions of computational thinking, their essential elements all include a set of concepts and thought processes that help learners in formulating, analyzing, and representing so as to solve problems (e.g., Barr & Stephenson, 2011; Mannila et al., 2014).

Partly, it means that CT is a way of thinking about how to solve problems (Lin & Liu, 2012; Robertson, 2011) and their solutions can be represented as computational steps and algorithms (Aho, 2012). Many researchers (e.g., Barr & Stephenson, 2011; Mannila et al., 2014) have argued that computer programming is an effective way to train learners to think like a computer scientist. Therefore, they believe that solving programming problems could promote the development of students' CT.

Students' Difficulties Solving Programming Problems

When solving programming problems, CT consists of three dimensions, namely, computational concepts, computational practices, and computational perspectives (Brennan & Resnick, 2012). Computational practices describe the processes and the adopted strategies of thinking and learning construction, and focus on the method of applying concepts in practice, including being incremental and iterative, testing and debugging, reusing and remixing, and abstracting and modularizing (Brennan & Resnick, 2012). However, most of these strategies rely on the programming environment or tools such as Visual Studio 2022 or CodeBlocks to plan, monitor, and regulate some important factors. Students regard programming as a trial-and-error process (Machuca Vivar et al., 2021) due to the immediate feedback from the programming environment rather than thinking deeply and coherently about a complete

algorithm. Teachers usually also help students' computational practices through the visualization of programming codes or they coach students closely and provide immediate feedback (Fessakis et al., 2013). These facts and efforts indicate that learners commonly encounter difficulties when solving programming problems.

The Teaching of Computational Thinking and Problem Solving

An important part of teaching computational thinking is to help learners find appropriate models of computation with which to formulate problems and derive solutions. However, the teaching of computational thinking mostly focuses on activities that promote the use of concepts of programming (e.g., Bers et al., 2014; Marcelino et al., 2018). Several studies have focused on approaches and strategies that help students formulate and solve problems. Acuna and Bansal (2021) established the Analysis-Design-Justification (ADJ) framework for the instruction of problem-solving skills; students responded positively to its use, and their problem-solving skills actually improved. According to Mladenovic et al.'s study (Mladenovic et al. 2020), when using block-based programming languages such as Scratch, students tend to achieve statistically significantly higher scores because these programming tools reduce students' cognitive load and allow them to focus on the logic and structures rather than on the syntax of writing programs. Their studies suggested that learners showed a better understanding of formulating and solving problems through appropriate models of computation.

In summary, approaches or strategies that help students formulate and solve problems and reduce cognitive load may facilitate the teaching of computational thinking. Moreover, students' approaches and strategies used in problem solving are related to their metacognition. Previous studies (Denny et al., 2019; Kapa, 2007) indicated that metacognition plays an important role in students' problem-solving performance, and students rely on their metacognition to solve programming problems (Cakiroglu & Betul, 2020). Therefore, metacognitive abilities are crucial for CT to guide students in selecting thinking strategies, planning and evaluating thought processes, and breaking down complex actions into conditional sequences.

Metacognition and Computational Thinking

Mayer (1998) stated that metacognitive knowledge involves the knowledge of when to use, how to coordinate, and how to monitor various skills in problem solving. In other words, metacognition involves the knowledge of cognition and of regulation of cognition (e.g., Brown, 1987; Flavell, 1979; Mayer, 1998). Teaching methods that promote students' metacognition include asking reflective questions (Mayer, 1998) and holding conditional knowledge discussions (Schraw, 1998). Some new strategies have also been found in recent years, such as encouraging students' monitoring and control of learning (Stanton et al., 2021).

Literature from computer education has pointed out that students' programming performance is affected by metacognition. For instance, [Chen et al. \(2021\)](#) found that metacognitive prompts (e.g., What specific strategies do you want to use to solve the problem?) have a more positive effect than cognitive prompts (e.g., What comes next to button switch X1 = PB1 and Y1 = A+?) on promoting vocational high school students' computational thinking when learning about programmable logic controllers. [Rum and Ismail \(2016\)](#) examined the metacognitive awareness inventory to identify how it relates to student programming achievement. Their findings indicated a positive relationship between metacognitive awareness and the learning success of introductory programming.

To assess computational thinking processes, [Allsop \(2019\)](#) evaluated the learning process in a computer game design context by interviewing and recording learners' activity when solving problems. She reported the metacognitive practices students used for controlling and regulating programming. Her research indicates the important role that metacognition plays in computational thinking. To move forward, experimental-based research may test the types of metacognitive strategies that could promote computational thinking. The present study, therefore, sought to address this gap.

Scaffolding for Metacognition during Computing

Scaffolding strategies involve related practices that support students as they learn and have been focused on the development of computational thinking in some studies ([Tikva & Tambouris, 2021](#)). Tikva and Tambouris summed up several types of scaffolding strategies from 12 studies on computational thinking. They are instructional scaffolding, guidance, adaptive, peer-, and resource-scaffolding. Scaffolding is also often divided into cognitive scaffolding, such as supporting understanding computational concepts ([Tikva & Tambouris, 2022](#)) and metacognitive scaffolding; for example, students are required to consider the strategies or methods to be used and the principles on which the strategies are based ([Chen et al., 2021](#)). Among them, some studies have explored the role of metacognition in computing teaching.

The literature has shown that when learners face metacognitive challenges during complex problem-solving processes, scaffolding is often needed to facilitate metacognitive thinking in various domains (e.g., [Denny et al., 2019](#); [Rum & Ismail, 2017](#)). Studies have also indicated that metacognitive training and support could accelerate students' understanding of programming (e.g., [Cetin et al., 2014](#); [Hauswirth & Adamoli, 2017](#)). Moreover, to help students avoid programming by trial and error in the programming environment whenever possible, scaffolding should be developed to support students' metacognition to facilitate solving programming problems.

[Abdul-Rahman and du Boulay \(2014\)](#) suggested that scaffolding for metacognition, such as question prompts and modeling strategies, could help students reflect upon their understanding. Studies have developed metacognitive support such as questioning ([Chetty & van der Westhuizen, 2014](#)), providing real-time feedback to students about their metacognition ([Beck et al., 2018](#)), collaborative discussion ([Bernard & Bachu, 2015](#))

and self-assessment (Rum & Ismail, 2017) for learning programming. Chetty and van der Westhuizen designed metacognition scaffolding to support first-year university students in making their programming plans. They used metacognitive strategy cards with questioning prompts such as “Am I able to describe the problem?” “Am I able to describe a solution to the problem?” and “Which programming constructs are needed to solve the problem?” The students designed the program after peer questioning with the prompts, as a result of which their overall success in programming improved.

Furthermore, Denny et al. (2019) reported that students who misinterpret a problem statement would most likely form an invalid mental model of the problem that they may find difficult to correct. Therefore, metacognitive scaffolding that requires deliberate reflection on the problem statement is important before any code is written. Chen et al. (2021) found that reasonable metacognitive prompts significantly improved students’ CT outcomes, self-efficacy, and motivation for learning, and found that reasonability does not always mean complexity. From this, easy-to-learn scaffolding, such as modeling strategies, should be designed for them.

Research Purposes

In sum, students have difficulties in programming, which is related to how to plan programming, namely, metacognitive planning. There have been studies that help students solve the problem of programming difficulties through metacognitive support. However, there is a lack of in-depth research on pre-programming metacognitive planning. To bridge this gap, the purposes of the current study were twofold. First, in study 1, we wanted to understand students’ programming planning before their programming practice and the relationship between students’ performance in programming planning and programming outcomes. If there are significant relationships, they can be used as the basis for scaffolding design. Second, in study 2, we designed two types of scaffolding comprising different planning steps to help students solve problems based on the results of study 1, and explored whether scaffolding involving fewer steps achieves the same learning outcome as scaffolding that entails more steps. Students’ learning efficiency may be improved by teachers adopting the former approach to support their students’ learning.

Method of Study I

Participants

The research data for study 1 were collected from 21 first-year university students (10 males and 11 females). Their average age was 18.54 (SD = .72). They came from a science major class who were learning the module, “C language Programming,” in Beijing. They were in the top 5% of the cohort of China high school graduates. They had no prior C language programming experience.

Instruments

In study 1, students were asked to write out their thought processes for the purpose of designing problem-solving solutions through programming before using the programming platform Dev C++; that is, they were asked to describe their programming planning. The written-down programming planning represented their metacognitive planning which was scored according to evaluation criteria (see Table 1) adopted from a review paper (Lye & Koh, 2014). Similar criteria have been widely applied in related research. For instance, Jing et al. (2020) used a mapping table to classify students'

Table 1. The Coding Scheme of Metacognitive Planning Performance.

Categories (Lye & Koh, 2014, p. 58)	Subcategories (Lye & Koh, 2014, p. 58)	Students' Metacognitive Planning Performance	Examples of Students' Statements (Taking the Problem "Constructing a Yang Hui Triangle" as an Example)
Understand the problem	1. Describe the problem	Find out the problem in the question, semantic understanding of the question	The Yang Hui triangle is a matrix problem
	2. Syntactic and semantic knowledge	Syntactic and semantic knowledge for specific program knowledge	Defines an array of 10 rows and 10 Columns $a[10][10]$
	3. Program comprehension	Select the right knowledge for known conditions	Using arrays and loops to solve the matrix problem
	4. Causal reasoning	Provide an effective solution to the problem	The numbers in the first column and diagonal are all 1, which need to be specially defined
Devise a plan	5. Find the location bug	Find out the point in the question (for complex questions)	The key problem is the connection between the numbers
	6. Make a connection between the programming scripts	Connect different parts of scripts such as "function"	None
	7. Causal reasoning	Difficulty in programming	The loop statement should be $a[i][j] = a[i-1][j-1] + a[i-1][j]$
Carry out the plan	8. Fix the bug	Run the program	—
Review the plan	9. Test the solution	Review of the result	—

Note. Lye and Koh (2014) provided nine subcategories. The researchers coded the statements of the thought processes students wrote down before programming according to the first seven subcategories because the latter two occurred at the beginning of and during programming.

metacognition capacity (e.g., planning, monitoring, evaluation, self-questioning) according to computational practices (e.g., imitation, iterating, modularizing, debugging) and behaviors (e.g., collaboration, communication, perseverance) in the programming process; Cakiroglu and Betul (2020) created a table with similar categories to evaluate students' metacognitive strategy development when solving programming problems; and Chen et al. (2021) formulated several metacognitive prompts for each component of CT to promote vocational high school students' CT skills in learning about programmable logic controllers, which are basically consistent with the student metacognition performance described in the coding scheme. According to Lye and Koh's (2014) criteria, the programming processes were split into nine steps corresponding to the processes of problem solving. The first seven steps correspond to the design of programming solutions. Compared to similar studies, some crucial steps such as defining the data structure have received extra attention (Lye & Koh, 2014). It is an important part of "algorithm design" (Chen et al., 2021) and could greatly affect the final result. For this study, the steps were designed to correspond to specific behaviors or results rather than some fuzzy concepts like "analysis" (Jing et al., 2020); therefore, our criteria could combine students' metacognition with program design to realize the measurement of the metacognition process in programming.

Students' metacognition planning scores were evaluated according to whether they gave the right statements, and either a 1 or a 0 was allocated according to the sub-categories of the framework. Table 1 illustrates an example of statements that received a score of 1. For example, one student stated the following: *Yang Hui's triangle¹ is a matrix problem*. This statement provides evidence of how he described the problem of "constructing a Yang Hui triangle," so he would get 1 point for "Describe the problem."

Students' CT was measured by a semester examination that was held on a software system. Students were asked to correct 12 program faults for a total score of 20 points, which was scored by the system according to built-in standard answers.

Data Collection and Analysis

In the first part, students' metacognitive planning and computational thinking performance were tested to explore which kinds of metacognitive planning steps are related to computational thinking. Students were given a programming task related to a real-life problem in the class to practice programming after learning basic knowledge once each week. Students were asked to write out their metacognitive planning before programming. Four guiding questions help students to articulate metacognitive planning. They were shown on the paper with the programming problem. "What concepts have you come up with according to the conditions of the problem?" "What are your thinking steps while designing the program?" "What strategies will you choose when you meet specific problems (such as the calculation of leap year)?" "What knowledge will you use when you meet specific programming tasks?" After writing the programming plan, they edited, revised, and debugged their programming solutions on the Dev C++ platform and turned in their assignments by the deadline. Students

practiced metacognitive planning for 6 weeks. Data were retained for three out of the six weekly tests, because the completion rate of the first 2 weeks was poor and several students were absent in another week.

After 6 weeks of practice, students' CT was measured. We adopted the delayed measure because we speculated that students would be able to develop an awareness of programming planning during a period of practice.

The Pearson correlation analysis was used to analyze students' metacognitive planning total performance and CT performance. The metacognitive planning overall performance was represented by the total score of the first seven subcategories in Table 1, which ranges from 0 to 7. Furthermore, the Pearson correlation of every step of metacognitive planning performance and CT performance was also analyzed.

Results of Study I

First, the distribution of data was tested. The skewness of students' CT performance (Mean = 17.27, SD = 3.14) is -1.08 , and the kurtosis is $-.07$ (Noar, 2003), which is a normal distribution. Then, to test the relationship between metacognitive planning total performance and computational thinking (CT) performance, Pearson correlation analysis was carried out (see Table 2). The metacognitive planning in Table 2 is the score of metacognitive planning total performance. As shown in Table 2, all 3 weeks of metacognitive planning were significantly positively correlated with students' computational thinking performance.

To explore what kinds of metacognitive planning steps work for computational thinking, Pearson correlation between each step of metacognitive planning (see Table 1, e.g., program comprehension) and computational thinking performance was conducted. As shown in Table 2, the step/sub-category of *describe the problem* was significantly positively correlated with students' computational thinking performance in week 1 and week 3; and the step/subcategory of *program comprehension*, was significantly positively correlated with students' computational thinking performance in all 3 weeks.

The results informed us which metacognitive planning activities are highly associated with programming performance. Moreover, the results are consistent with Kwon's (2010) findings that the scaffolding provided to students should focus on the

Table 2. Correlation between Metacognitive Planning and Computational Thinking.

	Computational Thinking		
	Week 1	Week 2	Week 3
Metacognitive planning	.52*	.41*	.50*
1. Describe the problem	.71*	.32	.43*
2. Program comprehension	.57*	.25*	.64**

Note. * $p < .05$, ** $p < .01$.

representation of problems and how to solve them. Fitzgerald et al.'s (2008) study also suggested that "Program comprehension" was considered a key element of successful programming. However, as correlations do not imply causation, scaffolding other aspects of metacognitive planning could also enhance programming performance. Hence, we designed two kinds of scaffolding to compare their effects on students' CT performance in the second part.

Method of Study 2

Participants

One year later, the data for study 2 were collected from another two classes ($N = 97$) of first year science majors in the same university for the same module, "C language Programming". The students were randomly assigned to three groups. After eliminating the missing data due to the occasional absence of eight students, 89 students' (35 males and 54 females) data were processed.

Scaffolding

Two types of scaffolding were designed, full-scaffolding (see Table 3) and semi-scaffolding (see Table 4). Full-scaffolding consisted of all the metacognitive planning subcategories, whereas semi-scaffolding consisted of the exact subcategories correlated with computational thinking performance. To simplify the learning process, students directly obtained hints and examples instead of being asked questions and discussing. Some professional vocabulary in the metacognitive planning subcategories, such as "program comprehension," was difficult for students to understand, so it was modified to help students understand more easily and is listed in the central column. *Find the real-world problem* actually asks students to *describe the problem* in a real-life way.

Based on the result of the correlation analysis in study 1, we designed a semi-scaffolding structure. To enhance the efficiency of learning, just a few crucial prompts are needed (Ge & Land, 2003). As a result, semi-scaffolding only consists of "describe the problem" and "problem comprehension"; these two subcategories are more significant and are correlated with CT performance. An illustration can be seen in Table 4.

Instruments

Students' metacognitive planning was scored with the same criteria as study 1. For CT performance, we assigned a programming problem for students every week; for example, "How can you calculate which day of the year a selected date is?" After writing out their metacognitive planning, students were required to write down their code on paper to avoid being debugged and revised with the help of the programming platform. The total score of a programming problem was 10 points, and every problem was graded according to the knowledge points and the related algorithm contained in the

Table 3. Full-Scaffolding (the Central and Right Columns) and Corresponding Subcategories (the Left Column).

Subcategories	Modified Subcategories	Please Write down Your Thought Process Referring to the Example Below
1. Describe the problem	Find the real-world problem	For example: 1. Input the year, month and day; 2. Different months have a different number of days; 3. Feb in a leap year is special
2. Syntactic and semantic knowledge	Syntactic and semantic knowledge you will use	For example: Use the "scanf" statement to input year, month, and day
3. Program comprehension	Select the program method	For example: 1. Use the "if" statement to judge if it is a leap year. 2. Use "switch" to select month
5. Locate the problem	Key problem needing to be solved	For example: 1. How to determine whether a year is a leap year; 2. Select a month
6. Connection between the programming scripts	Connection between the programming scripts, such as "function"	None
4/7. Causal reasoning	The way to solve the problem	For example: The code is "if((y%4==0&& y%100!=0) y%400==0)"

Table 4. Semi-Scaffolding (the Central and Right Columns) and Corresponding Subcategories (the Left Column).

Subcategories	Modified Subcategories	Please Write down Your Thought Process Referring to the Example Below
1. Describe the problem	Find the real-world problem	For example: 1. Input the year, month and day; 2. Different months have a different number of days; 3. Feb in a leap year is special
2. Program comprehension	Select the program method	For example: 1. Use the "if" statement to judge if it is a leap year; 2. Use "switch" to select month

solution. After every test, students' programming code was rated by two researchers back-to-back to verify the reliability. First, they determined the scoring principle based on two cases. Second, some samples were used to unify the standard. And then, other materials were coded on their own. Their consistency coefficients are .91, .84, .89, and .91, respectively, based on Spearman correlation analysis, indicating that their rating was effective.

Table 5. Correlation between Metacognitive Planning and Computational Thinking.

Week 1	Week 2	Week 3	Week 4
.56*	.76**	.63**	.69*

Note. * $p < .05$, ** $p < .01$.

Data Collection and Analysis

The experimental research method was adopted to compare the effects of the two kinds of scaffolding on improving CT. The experiment lasted for 4 weeks. In the first week, all participants were asked to do the same activities as study 1 to help students understand and familiarize themselves with metacognition without prompts. Before this, the teacher used an example (“Enter three integers, a, b, and c by the keyboard and output the largest of them.”) and the think-aloud problem-solving process as an example to explain how to do metacognitive planning. In the right column of Table 3, examples of students’ solutions to the problem “How can you calculate which day of the year a selected date is?” are provided. Students had already solved this problem in the first week of the experiment. From the second week, students were randomly divided into three groups. They wrote metacognitive planning with the different types of scaffolding. Group 1 used the full-scaffolding approach, group 2 used semi-scaffolding, and group 3 had no scaffolding.

Due to the small sample of study 1, Pearson correlation between metacognitive planning and CT in 4 weeks was analyzed again to verify the results of study 1. Then, students’ CT growth curves of the three groups in 4 weeks were constructed. Finally, one-way analysis of covariance (ANCOVA) was applied to examine the effects of the different types of scaffolding with LSD method for post hoc. The grade in week 1 was employed as the pretest and the covariates, and the grade in week 4 was used as the posttest and the dependent variables.

Results of Study 2

Table 5 shows significant correlation between metacognitive planning overall performance and computational thinking (CT) performance. The results of study 1 were confirmed again.

The changes in students’ CT and the differences among the three groups are illustrated in Figure 1. As shown in Figure 1, the performances of the three groups of students were almost the same initially. Differences between the groups emerged in week 2 and appeared to stabilize in weeks 3 and 4.

ANCOVA was applied to explore the effects of the different types of scaffolding on the participants’ CT. The assumption of homogeneity of regression slopes was tested and was not violated ($F = .01, p > .05$). As shown in Table 6, significant differences were found between the three groups in their computational thinking ($F = 4.07, p < .05$).

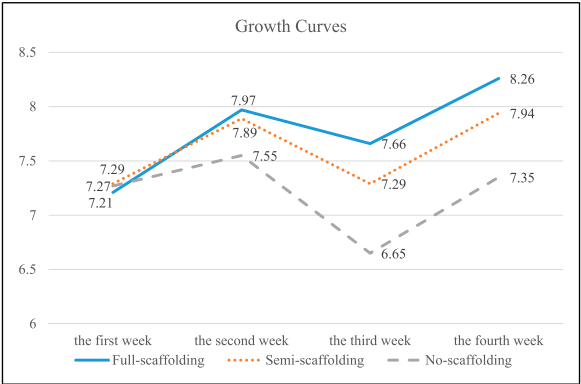


Figure 1. Students’ CT growth curves of the three groups over 4 weeks.

Table 6. Description Data, ANCOVA on the Computational Thinking Posttest Result.

Post-test Variables	N	Mean	SD	Adjusted Mean	F Value	Post Hoc
(1) Full-scaffolding group	29	8.24	1.55	8.21	4.07*	(1) > (2) sig. = .313
(2) Semi-scaffolding group	31	7.94	1.37	7.97		(1) > (3) sig. = .033*
(3) No-scaffolding group	29	7.35	1.54	7.27		(2) > (3) sig. = .018*

Note. *p < .05.

When the mean scores between any two groups were compared, it was found that students with scaffolding (including full-scaffolding and semi-scaffolding) seemed to have better computational thinking performance (see Table 6). It can be inferred that scaffolding improves students’ metacognitive planning and enhances their problem solving through programming, namely, computational thinking. However, no significant difference was found between the full-scaffolding group and the semi-scaffolding group. It seems that students’ computational thinking performance with semi-scaffolding approached a similar level to the students’ performance with full-scaffolding.

Discussion

Previous studies (e.g., Denny et al., 2019; Kapa, 2007) suggested the important role metacognition plays in problem solving. Computational thinking involves using computer algorithms to facilitate students’ problem solving (Mannila et al., 2014). The current study tested the quantitative relationship between metacognitive planning and computational thinking. Subsequently, two kinds of scaffolding were developed to support students’ metacognitive planning based on the results of study 1. The results drawn from study 2 suggested that two kinds of scaffolding could help their

computational thinking, and the semi-scaffolding with two steps may help teachers and students focus on the more critical steps to save time and promote efficiency.

Metacognitive Planning and Computational Thinking

Study 1 of this study was an initial attempt to explore the students' metacognitive planning in computational thinking through a quantitative method. The positive correlations between metacognitive scaffoldings and programming performance indicated that scaffolding might help cultivate computational thinking. Consistent with previous research (Allsop, 2019), the findings indicated that metacognitive planning was an important factor influencing computer programming learning. Hence, teachers could improve students' computational thinking from the perspective of metacognition. More importantly, in response to research question 1, the "Describe the problem" and "Program comprehension" subcategories in all the metacognitive planning steps were significantly related to the performance of problem solving through programming, that is, computational thinking. Previous research (e.g., Fitzgerald et al., 2008; Kwon, 2010) also agreed that "Describe the problem" during problem solving and "Program comprehension" were considered to be the key elements for successful programming. The findings indicated that the representation of problems, corresponding to identifying the problem in the question and semantic understanding of the question is important. In addition, selecting the right knowledge for known conditions is crucial to understanding programming. These might be the major difficulties students need to overcome.

Researchers have pointed out that the main principle of the instructional approach is to support metacognitive ability (Rum & Ismail, 2014). Because students have difficulties with metacognition, especially for problem solving involving multiple steps and conditions, teachers can provide students with some support, such as scaffolding, to help them focus on problematic aspects to overcome the difficulties and improve their learning achievement (Bannert & Mengelkamp, 2013; Valencia-Vallejo et al., 2019). Consequently, two kinds of scaffolding for metacognitive planning were developed. Full-scaffolding consists of six steps in the framework, whereas the semi-scaffolding approach only consists of "Describe the problem" and "Program comprehension" according to the correlation analysis result.

Most studies have focused on metacognition with CT (e.g., Chen et al., 2021; Chetty & van der Westhuizen, 2014) or computational practices (e.g., Jing et al., 2020), whereas our study not only uncovered the relationship between metacognitive planning and CT but also identified concrete planning steps. The study is similar to Chetty and van der Westhuizen's, which developed the metacognitive strategy card, including students' planning strategies support. Furthermore, our study took the coding scheme reviewed by Lye and Koh (2014) as a theoretical basis. The scheme clarifies how to understand the problem and devise a plan in seven steps while programming. This is a key supporting the study exploring the relationship between both variables and the effect of different types of scaffolding.

Effect of Different Types of Scaffolding

The data from [Table 5](#) answer the second research question and show that the implementation of scaffolding in metacognitive planning may benefit students' computational thinking. Consistent with the findings of some research (e.g., [Chen et al., 2021](#); [Chetty & van der Westhuizen, 2014](#)), supporting metacognition may improve computational thinking. Accordingly, teachers should consider metacognition support in their teaching design to promote students' computational thinking.

For the second question, the data also provide empirical evidence that the semi-scaffolding, which only includes "Describe the problem" and "Program comprehension," has a similar function to the full-scaffolding approach. Semi-scaffolding with key steps may be more directed and hence reduce unnecessary extraneous cognitive load compared with seven-step full-scaffolding. The smaller the extraneous cognitive load is, the more relevant information elements can be processed in the same time unit, and the less information will be lost ([Van Gerven et al., 2002](#)). Scaffolding based on learners' weakness is likely to promote effective learning. The effect of semi-scaffolding aligns with the results that students may need only a few prompts to solve problems ([Ge & Land, 2003](#)). Other evidence also supports that reducing cognitive overload by appropriate scaffolding is an effective way to teach programming courses ([Reardon & Tangney, 2015](#)). Accordingly, while constructing scaffolding for computational thinking, teachers should identify students' weaknesses and employ targeted strategies that help develop students' metacognition and then reduce their extraneous cognitive load. Instead of visualizing the programming code ([Mladenovic et al., 2020](#)) or using question cards ([Chetty & van der Westhuizen, 2014](#)), our scaffolding used written prompts to support students' metacognition development. Both the ways of giving prompts and hints or designing scaffolding based on students' weakness reduce students' extraneous cognitive load. The metacognitive planning steps of "Describe the problem" and "Program comprehension" in our scaffolding might help students decompose the problems and further seek solutions from the perspective of computational thinking.

Although previous studies have found that metacognition plays an important role in computational thinking, our research provides empirical evidence of the kinds of specific metacognitive planning steps that are effective for promoting computational thinking. This study has also confirmed the significance of key-point structure scaffolding in improving students' computational thinking. The study provides a direction for the optimization of metacognitive scaffolds, which may lay an empirical basis for teachers' guidance in programming education. It is hoped that the findings of this study will contribute to the design of more efficient metacognitive scaffolding for teaching computational thinking to enhance the teaching effect and improve the development level of students' CT.

Conclusion

From two quantitative studies, we found that the metacognitive planning performance related to "problem description" and "program comprehension" was significantly

correlated with programming problem-solving performance; the no-scaffolding group was significantly weaker than the other two groups with scaffolding, but there was no significant difference between the semi-scaffolding and the full-scaffolding groups. The results illustrate that metacognitive planning scaffolding for problem solving in programming is sufficient and efficient for the cultivation of students' computational thinking. The similar effects of full-scaffolding and semi-scaffolding suggest that students' "problem description" and "program comprehension" are the aspects that may need scaffolding.

To sum up, this research specifically examined students' metacognition planning before solving programming problems and the specific aspects of scaffolding needed for metacognition planning. Hence, compared with previous research that aimed to support students' programming, the research identified two specific aspects of scaffolding for metacognition planning that are supportive of CT. The study highlights the role students' metacognition planning plays in programming problem solving and CT training. [Lye and Koh's \(2014\)](#) framework clarified the steps of programming problem solving, and we have successfully applied it to support students' learning. Our findings may point out how parsimonious scaffolding could be provided. Nonetheless, the findings need to be further verified with larger sample size and perhaps with more nuanced scaffolding for these two dimensions.

Limitations and Suggestions

In this study, scaffolding was used to support students' metacognitive planning and solve the problem. Although scaffolding may be more diversified and customized for individualized learning (e.g., [Kao et al., 2017](#)), it was not our main purpose to construct different types of scaffolding. Future studies should develop more specific steps of metacognitive planning and construct more types of scaffolding with different characteristics to fulfill different students' needs. Moreover, teachers should identify the key elements and simplify the construction when they design their own scaffolding. There is a need for applicable criteria to analyze metacognition content by quantitative approaches.

The small sample of study 1 is a limitation. The change in students' computational thinking was tested after 4 weeks of teaching intervention. Further studies may need to be implemented for longer periods of time. Researchers should further examine whether metacognitive planning would change students' computational concepts, practices, and perspectives with more participants.

According to the research results, we also suggest that teachers elaborate "Describe the problem" and "Program comprehension" when analyzing a program in the course. Furthermore, semi-scaffolding support should be provided when students need to develop a program.

Declaration of Conflicting Interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This project was funded by the Beijing Social Science Foundation Project (22JYA005).

ORCID iDs

Ying Zhou  <https://orcid.org/0000-0001-5736-2094>

Jyh-Chong Liang  <https://orcid.org/0000-0002-2423-5950>

Note

1. Yang Hui's triangle refers to a geometric arrangement of binomial coefficients in a triangle. It first appeared in the work "Mathematical Methods Explaining in Detail the Nine Chapters" (Xiangjie jiuzhang suanfa 详解九章算法, CE 1261) written by Yang Hui, a Chinese mathematician in the Southern Song Dynasty. In Europe, Pascal first discovered this rule in 1654, so the geometric arrangement is known in Western countries as Pascal's triangle. The students' task was to output at least 10 lines of Yang Hui's triangle by programming.

References

- Abdul-Rahman, S. S., & du Boulay, B. (2014). Learning programming via worked-examples: Relation of learning styles to cognitive load. *Computers in Human Behavior*, 30(1), 286–298. <https://doi.org/10.1016/j.chb.2013.09.007>
- Acuna, R., & Bansal, A. (2021). Analysis-Design-Justification (ADJ): A framework to develop problem-solving skills. Proceedings of the 2021 IEEE Global Engineering Education Conference (EDUCON) (pp. 372–378). <http://doi.org/10.1109/EDUCON46332.2021.9454055>
- Aho, A. V. (2012). Computation and computational thinking. *The Computer Journal*, 55(7), 832–835. <http://doi.org/10.1093/comjnl/bxs074>
- Allsop, Y. (2019). Assessing computational thinking process using a multiple evaluation approach. *International Journal of Child-Computer Interaction*, 19, 30–55. <https://doi.org/10.1016/j.ijcci.2018.10.004>
- Bannert, M., & Mengelkamp, C. (2013). Scaffolding hypermedia learning through metacognitive prompts. *International handbook of metacognition and learning technologies* (pp. 171–186). Springer. https://doi.org/10.1007/978-1-4419-5546-3_12
- Barr, V., & Stephenson, C. (2011). Bringing computational thinking to K-12: What is involved and what is the role of the computer science education community? *ACM Inroads*, 2(1), 48–54. <https://doi.org/10.1145/1929887.1929905>

- Beck, P. J., Mohammadi-Aragh, M. J., Archibald, C., Jones, B. A., & Barton, A. (2018). Real-time metacognition feedback for introductory programming using machine learning. 2018 IEEE Frontiers in Education Conference (FIE) (pp. 1–5). <https://ieeexplore.ieee.org/document/8658973>
- Bernard, M., & Bachu, E. (2015). Enhancing the metacognitive skill of novice programmers through collaborative learning. *Intelligent Systems Reference Library*, 76, 277–298. https://doi.org/10.1007/978-3-319-11062-2_11
- Bers, M. U., Flannery, L., Kazakoff, E. R., & Sullivan, A. (2014). Computational thinking and tinkering: Exploration of an early childhood robotics curriculum. *Computers and Education*, 72, 145–157. <http://doi.org/10.1016/j.compedu.2013.10.020>
- Brennan, K., & Resnick, M. (2012). New frameworks for studying and assessing the development of computational thinking. Proceedings of the 2012 Annual Meeting of the American Educational Research Association (pp. 1–25). <https://www.mendeley.com/catalogue/b0df8b57-0f03-3702-917f-18be4a0efb92/>
- Brown, A. L. (1987). Metacognition, executive control, self-regulation, and other more mysterious mechanisms. In F. E. Weinert & R. H. Kluwe (Eds) *Metacognition, motivation, and understanding* (pp. 65–116). Lawrence Erlbaum. <https://www.mendeley.com/catalogue/9cb2a47e-02c6-34fb-a84f-7f17a4199a33/>
- Cakiroglu, U., & Er, B. (2020). Effect of using metacognitive strategies to enhance programming performances. *Informatics in Education*, 19(2), 181–200. <http://doi.org/10.15388/infedu.2020.09>
- Cetin, I., Sendurur, E., & Sendurur, P. (2014). Assessing the impact of meta-cognitive training on students' understanding of introductory programming concepts. *Journal of Educational Computing Research*, 50(4), 507–524. <https://doi.org/10.2190/ec.50.4.d>
- Chen, C. H., Liu, T. K., & Huang, K. (2021). Scaffolding vocational high school students' computational thinking with cognitive and metacognitive prompts in learning about programmable logic controllers. *Journal of Research on Technology in Education*, 1–18. <http://doi.org/10.1080/15391523.2021.1983894>
- Chetty, J., & van der Westhuizen, D. (2014). Implementing metacognition skills for learners studying computer programming. In J. Viteli & M. Leikomaa (Eds) Proceedings of EdMedia 2014: World Conference on educational Media and technology (pp. 726–731). Association for the Advancement of Computing in Education (AACE). <https://www.learntechlib.org/primary/p/147775/>
- Denny, P., Prather, J., Becker, B. A., Albrecht, Z., Loksa, D., & Pettit, R. (2019). A closer look at metacognitive scaffolding: Solving test cases before programming. 19th Koli Calling Conference on Computing Education Research (KOLI CALLING 2019). <http://doi.org/10.1145/3364510.3366170>
- Fessakis, G., Gouli, E., & Mavroudi, E. (2013). Problem solving by 5-6 years old kindergarten children in a computer programming environment: A case study. *Computers and Education*, 63, 87–97. <http://doi.org/10.1016/j.compedu.2012.11.016>
- Fitzgerald, S., Lewandowski, G., McCauley, R., Murphy, L., Simon, B., Thomas, L., & Zander, C. (2008). Debugging: Finding, fixing and flailing, a multi-institutional study of novice

- debuggers. *Computer Science Education*, 18(2), 93–116. <https://doi.org/10.1080/08993400802114508>
- Flavell, J. H. (1979). Metacognition and cognitive monitoring: A new area of cognitive–developmental inquiry. *American Psychologist*, 34(10), 906–911. <https://doi.org/10.1037/0003-066X.34.10.906>
- Ge, X., & Land, S. M. (2003). Scaffolding students' problem-solving processes in an ill-structured task using question prompts and peer interactions. *Educational Technology Research and Development*, 51(1), 21–38. <http://doi.org/10.1007/BF02504515>
- Hauswirth, M., & Adamoli, A. (2017). Metacognitive calibration when learning to program. 17th Koli Calling International Conference on Computing Education Research (Koli Calling 2017) (pp. 50–59). <http://doi.org/10.1145/3141880.3141904>
- Jing, S., Tang, Y., Liu, X., & Gong, X. (2020). A learner model integrating cognitive and metacognitive and its application on scratch programming projects. *IFAC-PapersOnLine*, 53(5), 644–649. <http://doi.org/10.1016/j.ifacol.2021.04.154>
- Kao, G. Y. M., Chiang, C. H., & Sun, C. T. (2017). Customizing scaffolds for game-based learning in physics: Impacts on knowledge acquisition and game design creativity. *Computers and Education*, 113, 294–312. <http://doi.org/10.1016/j.compedu.2017.05.022>
- Kapa, E. (2007). Transfer from structured to open-ended problem solving in a computerized metacognitive environment. *Learning and Instruction*, 17(6), 688–707. <http://doi.org/10.1016/j.learninstruc.2007.09.019>
- Kwon, S. (2010). Research on cognitive load theory and its design implications for problem solving instruction. *Educational Technology International*, 11(1), 93–117. <https://koreascience.kr/article/JAKO201014436705857.pdf>
- Lin, J. M., & Liu, S. F. (2012). An investigation into parent-child collaboration in learning computer programming. *Educational Technology and Society*, 15(1), 162–173. https://drive.google.com/open?id=1v8KqJHOIMq2b_I3G-WwjW3LCmMk-ubvC
- Lye, S. Y., & Koh, J. H. L. (2014). Review on teaching and learning of computational thinking through programming: What is next for K-12? *Computers in Human Behavior*, 41, 51–61. <http://doi.org/10.1016/j.chb.2014.09.012>
- Machuca Vivar, S. A., Sampedro Guamán, C. R., Palma Rivera, D. P., & Cañizares Galarza, F. P. (2021). Desarrollo de la lógica de programación en estudiantes de sistemas de Uniandes Santo Domingo. *Conrado*, 17(79), 214–224. <https://doaj.org/article/bf487bdf68ab4c44b7e45a22f53d4959>
- Mannila, L., Dagiene, V., Demo, B., Grgurina, N., Mirolo, C., Rolandsson, L., & Settle, A. (2014). Computational thinking in K-9 education. Proceedings of the Working Group Reports of the 2014 on Innovation & Technology in Computer Science Education Conference (pp 1–29). ACM. <https://doi.org/10.1145/2713609.2713610>
- Marcelino, M. J., Pessoa, T., Vieira, C., Salvador, T., & Mendes, A. J. (2018). Learning computational thinking and scratch at distance. *Computers in Human Behavior*, 80, 470–477. <http://doi.org/10.1016/j.chb.2017.09.025>
- Mayer, R. E. (1998). Cognitive, metacognitive, and motivational aspects of problem solving. *Instructional Science*, 26(1/2), 49–63. <https://doi.org/10.1023/A:1003088013286>

- Mladenovic, M., Žanko, Ž., & Mladenović, S. (2020). Impact of used programming language for K-12 students' understanding of the loop concept. *International Journal of Technology Enhanced Learning*, 12(1), 79–98. <http://doi.org/10.1504/IJTEL.2020.10024760>
- Noar, S. M. (2003). The role of structural equation modeling in scale development. *Structural Equation Modeling: A Multidisciplinary Journal*, 10(4), 622–647. https://doi.org/10.1207/S15328007SEM1004_8
- Reardon, S., & Tangney, B. (2015). Smartphones, studio-based learning, and scaffolding: Helping novices learn to program. *ACM Transactions on Computing Education*, 14(4), 1–15. <http://doi.org/10.1145/2677089>
- Robertson, J. (2011). The educational affordances of blogs for self-directed learning. *Computers and Education*, 57(2), 1628–1644. <http://doi.org/10.1016/j.compedu.2011.03.003>
- Rum, S. N. M., & Ismail, M. A. (2014). Ontology development of metacognitive support system for novice programmers (MSSNP). H. A. BinSulaiman (Ed.) 2014 1st International Symposium on Technology Management and Emerging Technologies (ISTMET 2014) 316–321. <https://doi.org/10.1109/ISTMET.2014.6936526>
- Rum, S. N. M., & Ismail, M. A. (2016). Metacognitive awareness assessment and introductory computer programming course achievement at university. *International Arab Journal of Information Technology*, 13(6), 667–676. <http://iajit.org/PDF/Vol.13,No.6/7804.pdf>
- Rum, S. N. M., & Ismail, M. A. (2017). Metacognitive support accelerates computer assisted learning for novice programmers. *Educational Technology and Society*, 20(3), 170–181. <https://drive.google.com/open?id=1UErRCLhahuyRCqY6esbHMrHVB3WPc8Wz>
- Schraw, G. (1998). Promoting general metacognitive awareness. *Instructional Science*, 26(1/2), 113–125. <http://doi.org/10.1023/A:1003044231033>
- Stanton, J. D., Sebesta, A. J., & Dunlosky, J. (2021). Fostering metacognition to support student learning and performance. *CBE-Life Sciences Education*, 20(2), fe3. <http://doi.org/10.1187/cbe.20-12-0289>
- Tikva, C., & Tambouris, E. (2021). Mapping computational thinking through programming in K-12 education: A conceptual model based on a systematic literature review. *Computers and Education*, 162, 104083. <https://doi.org/10.1016/j.compedu.2020.104083>
- Tikva, C., & Tambouris, E. (2022). *The effect of scaffolding programming games and attitudes towards programming on the development of Computational Thinking*. Education and Information Technologies. <http://doi.org/10.1007/s10639-022-11465-y>
- Valencia-Vallejo, N., Lopez-Vargas, O., & Sanabria-Rodriguez, L. (2019). Effect of a metacognitive scaffolding on self-efficacy, metacognition, and achievement in e-learning environments. *Knowledge Management and E-Learning-An International Journal*, 11(1), 1–19. <https://doi.org/10.34105/j.kmel.2019.11.001>
- Van Gerven, P. W. M., Paas, F. G. W. C., Van Merriënboer, J. J. G., & Schmidt, H. G. (2002). Cognitive load theory and aging: Effects of worked examples on training efficiency. *Learning and Instruction*, 12(1), 87–105. [https://doi.org/10.1016/S0959-4752\(01\)00017-2](https://doi.org/10.1016/S0959-4752(01)00017-2)
- Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, 49(3), 33–35. <http://doi.org/10.1145/1118178.1118215>

Author Biographies

Ying Zhou is an associate professor in Beijing Normal University. Her research interests include artificial intelligence in education, science and technology education, and visualization of knowledge and thinking.

Ching Sing Chai is currently a professor in the Chinese University of Hong Kong. His research interests include technological pedagogical content knowledge, language learning, STEM education, and teacher professional development.

Xiuting Li is a PhD candidate in Beijing Normal University.

Chao Ma is a Master of Science in educational technology in Beijing Normal University.

Baoping Li is an associate professor at Faculty of Education in Beijing Normal University. Her research interests deal with designing and assessment of smart learning environment, hybrid programming learning, and teacher professional development in technology enhanced learning environment.

Ding Yu is a graduate student in computer science in Beijing Normal University.

Jyh-Chong Liang is a Professor at the Program of Learning Sciences and Institute for Research Excellence in Learning Sciences in National Taiwan Normal University. His research interests deal with scientific epistemological beliefs, conceptions of and approaches to learning science, TPACK, and Web information assessment.